

Configuring Cucumber with Selenium

Now, let's look at how we can integrate Cucumber with Selenium for automated testing in DevOps.

The prerequisites are any IDE (I will be taking Eclipse Neon for tutorial purposes) and the latest Java installed on your system.

Jars required: The following jars/drivers need to be downloaded before starting with the configuration:

- Selenium-server-standalone (latest version)
- Selenium client for Java (latest version available)
- Cobertura-2.1.1 or above
- Cucumber-core-1.2.5 or above
- Cucumber-java-1.2.5 or above
- Cucumber-junit-1.2.5 or above
- Cucumber-jvm-deps-1.0.6 or above
- Cucumber-reporting-3.9.0 or above
- Gherkin-2.12.2
- Junit-3.4.2 or above
- Mockito-all-2.0.2-beta or above
- A driver corresponding to your browser (I will be using the Chrome driver)

After all the required files have been downloaded, follow the steps given below.

1. Launch Eclipse and create a Java project named 'CucumberSelenium' in it.
2. Create three packages under the `src` folder named `cucumberTest`, `resources.Files` and `testScripts`, which will each contain the `runner` file, `test case feature` file and the `step definition` file.

Feature file: This file will contain our test case written in Gherkin language and will have the *feature* extension.

Runner file: Our Cucumber-Selenium framework will not have the *Main* method since we will be using JUnit to run our Java class. So this Java class will be run as a JUnit test to run our script.

 **Note:** Each feature file will have its separate runner class in this framework.

Step definition file: We have designed our *feature* file and *runner* file, but how will Cucumber get to know what code to execute for the specific test step mentioned in the *feature* file? This is taken care of by a separate Java class called *step definition* file.

3. Now create the `CucumberRunner.java` class under the `cucumberTest` package, `testCase.feature` file under the `resources.Files` package and `Testcase.java` class under the `stepDefinition` package.
4. Next, copy the following in your runner class and feature file.

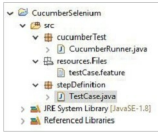


Figure 3: The file hierarchy in the project will look like this

For `CucumberRunner.java`, copy:

```
package cucumberTest;

import org.junit.runner.RunWith;
import cucumber.api.CucumberOptions;
import cucumber.api.junit.Cucumber;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/resources/Files/
    testCase.feature",
    glue = {"stepDefinition"},
    tags = {"@cucumber"}
)

public class CucumberRunner {
}
```

For `testCase.feature`, copy:

```
Feature: To check functionality of google search page

@cucumber
Scenario Outline:
    Given <required> browser is opened
    When <url> is opened
    And <keyword> is searched in search box
    Then the first link in search results should be
opened

    And browser is closed

Examples:
    | required | url | keyword |
    | "chrome" | "http://www.google.com" | "DevOps" |
```

5. Now run your *runner* file as a JUnit test, and you can see the empty methods being auto generated in the console output of Eclipse for each test step in the *feature* file. The significance of these methods is that Cucumber reads test steps from your *feature* file and searches for the corresponding method in the package mentioned in the *glue* option of the *runner* file. The variable values are also passed as arguments to the methods, to make use of them while scripting. So you can see how

```
1 Scenarios (8f13ae1 undefined)(0m)
5 Steps (8f13ae1 undefined)(0m)
0m.000s

You can implement missing steps with the snippets below:

@Given("^\"([^\"]*)\"") browser is opened"$)
public void browser_is_opened(String arg1) throws Throwable {
    // Write code here that turns the phrase above into concrete actions
    throw new PendingException();
}
```

Figure 4: Methods' skeleton being generated by Cucumber by parsing the *feature* file